

Functions in C

- In C, we can divide a large program into the basic building blocks known as function.
- The function contains the set of programming statements enclosed by { }.
- A function can be called multiple times to provide reusability and modularity to the C program.
- In other words, we can say that the collection of functions creates a program. The function is also known as *procedure* or *subroutine* in other programming languages.

Advantage of functions in C

- By using functions, we can avoid rewriting same logic/code again and again in a program.
- We can call C functions any number of times in a program and from any place in a program.
- We can track a large C program easily when it is divided into multiple functions.
- Reusability is the main achievement of C functions.
- However, Function calling is always a overhead in a C program.

The syntax of creating function in c language is given below:

```
return_type function_name(data_type parameter...){  
//code to be executed  
}
```

Types of Functions

There are two types of functions in C programming:

1. **Library Functions:** are the functions which are declared in the C header files such as `scanf( )`, `printf( )`, `gets( )`, `puts( )`, `ceil( )`, `floor( )` etc.
2. **User-defined functions:** are the functions which are created by the C programmer, so that he/she can use it many times. It reduces the complexity of a big program and optimizes the code.

Return Value

A C function may or may not return a value from the function. If you don't have to return any value from the function, use void for the return type.

Let's see a simple example of C function that doesn't return any value from the function.

```
void hello( )  
  
{  
  
printf("hello c");  
  
}
```

If you want to return any value from the function, you need to use any data type such as int, long, char, etc. The return type depends on the value to be returned from the function.

Let's see a simple example of C function that returns int value from the function.

Example with return value:

```
int get( )  
{  
    return 10;  
}
```

```
float get( )  
{  
    return 10.2;  
}
```

Function Aspects

There are three aspects of a C function.

- **Function declaration** A function must be declared globally in a c program to tell the compiler about the function name, function parameters, and return type.
- **Function call** Function can be called from anywhere in the program. The parameter list must not differ in function calling and function declaration. We must pass the same number of functions as it is declared in the function declaration.
- **Function definition** It contains the actual statements which are to be executed. It is the most important aspect to which the control comes when the function is called. Here, we must notice that only one value can be returned from the function.

SN	C function aspects	Syntax
1	Function declaration	return_type function_name (argument list);
2	Function call	function_name (argument_list)
3	Function definition	return_type function_name (argument list) {function body;}

Different aspects of function calling

A function may or may not accept any argument. It may or may not return any value. Based on these facts, There are four different aspects of function calls.

- function without arguments and without return value
- function without arguments and with return value
- function with arguments and without return value
- function with arguments and with return value

Example for Function without argument and return value

```
#include<stdio.h>
```

```
void printName( );
```

```
int main ()
```

```
{
```

```
    printf("Hello ");
```

```
    printName();
```

```
    return 0;
```

```
}
```

```
void printName()
```

```
{
```

```
    printf("Javatpoint");
```

```
}
```

Function Declaration

Function Call

Function Definition

Output

Hello Javatpoint

Example 2:

```
#include<stdio.h>
```

```
void sum( );
```

```
int main()
```

```
{
```

```
printf("calculate the sum of 2 numbers:");
```

```
    sum( );
```

```
    return 0;
```

```
}
```

```
void sum()
```

```
{
```

```
    int a,b;
```

```
    printf("Enter two numbers");
```

```
    scanf("%d %d",&a,&b);
```

```
    printf("The sum is %d",a+b);
```

```
}
```

Output

Going to calculate the sum of two numbers:

Enter two numbers 10

24

The sum is 34

Example 1: Example for Function without argument and with return value

```
#include<stdio.h>
```

```
int sum();
```

```
int main()
```

```
{  int result;
```

```
printf("calculate the sum of 2 numbers:");
```

```
    result = sum();
```

```
    return 0;
```

```
}
```

```
int sum()
```

```
{
```

```
    int a,b;
```

```
    printf("Enter two numbers");
```

```
    scanf("%d %d",&a,&b);
```

```
    return a+b;
```

```
}
```

Output

Going to calculate the sum of two numbers:

Enter two numbers 10

24

The sum is 34

Example 2: program to calculate the area of the square

```
#include<stdio.h>
```

```
int sum( );
```

```
int main( )
```

```
{
```

```
    printf(" area of the square\n");
```

```
    float area = square( );
```

```
    printf("Area of the square: %f\n",area);
```

```
    return 0;
```

```
}
```

```
int square()
```

```
{
```

```
    float side;
```

```
    printf("Enter the length of the side in  
meters: ");
```

```
    scanf("%f",&side);
```

```
    return side * side;
```

```
}
```


Output

Going to calculate the area of the square
Enter the length of the side in meters: 10
The area of the square: 100.000000

Example for Function with argument and without return value

```
#include<stdio.h>

void sum(int, int);

int main()
{
    int a,b,result;
    printf("calculate the sum of two numbers:");
    printf("Enter two numbers:");
    scanf("%d %d",&a,&b);
    sum(a,b);
    return 0;
}
```

```
void sum(int a, int b)
{
    printf("The sum is %d",a+b);
}
```

Output

Going to calculate the sum of two numbers:

Enter two numbers 10

24

The sum is 34

Example 2: program to calculate the average of five numbers.

```
#include<stdio.h>

void average(int, int, int, int, int);

void main()
{
    int a,b,c,d,e;
    printf(" the average of five numbers:");
    printf("\nEnter five numbers:");
    scanf("%d %d %d %d %d",&a,&b,&c,&d,&e);
    average(a,b,c,d,e);
}
```

```
void average(int a, int b, int c, int d, int
e)
{
    float avg;

    avg = (a+b+c+d+e)/5;

    printf("The average of given five numbers
: %f",avg);
}
```

Output

Going to calculate the average of five numbers:

Enter five numbers:10

20

30

40

50

The average of given five numbers : 30.000000

Example for Function with argument and with return value

```
#include<stdio.h>

int sum(int, int);

void main()
{
    int a,b,result;

    printf("the sum of two numbers:");

    printf("Enter two numbers:");

    scanf("%d %d",&a,&b);

    result = sum(a,b);

    printf("The sum is : %d",result);
}
```

```
int sum(int a, int b)
{
    return a+b;
}
```

Output

Going to calculate the sum of two numbers:
Enter two numbers:10
20
The sum is : 30

Example 2: Program to check whether a number is even or odd

```
#include<stdio.h>

int even_odd(int);

void main()
{
    int n,flag=0;

    printf("check whether a number is even or odd");
    printf("Enter the number: ");
    scanf("%d",&n);

    flag = even_odd(n);

    if(flag == 0)
    {
        printf("number is odd");
    }
}
```

```
else
{
    printf("number is even");
}
}

int even_odd(int n)
{
    if(n%2 == 0)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}
```

Output

Going to check whether a number is even or odd

Enter the number: 100

The number is even